Computing x_d and u_d from Waypoints with Minimum Snap

Purpose

This note summarizes how the current pipeline converts sparse RRT or LLM waypoints into desired states x_d and feedforward controls u_d using minimum-snap trajectories. It also compares the previous separate-clock scheduling against the newer shared-clock scheduling used for RRT-vs-LLM calibration scores.

The relevant implementation is in

`src/llm_vision_planner/fine_tuning/scripts/min_snap.py`.

Step 1: Assign Segment Durations

Given waypoints

$$w_0, w_1, \ldots, w_N,$$

each segment connects w_i to w_{i+1}. The default duration assignment is

$$T_i = \max\left(\frac{\|w_{i+1} - w_i\|_2}{v_{\text{cruise}}},\ 0.5\right), \qquad v_{\text{cruise}} = 0.5 \text{ m/s}.$$

This is why short LLM segments can get short durations while longer RRT segments get longer durations.

Step 2: Fit a Seventh-Degree Polynomial

For each segment and for each axis, the code uses a seventh-degree polynomial:

$$p_i(\tau) = c_{i,0} + c_{i,1}\tau + c_{i,2}\tau^2 + \cdots + c_{i,7}\tau^7, \qquad 0 \le \tau \le T_i.$$

This is done separately for $x(\tau)$ and $y(\tau)$. The derivatives are

$$\dot{p}_i(\tau) = c_{i,1} + 2c_{i,2}\tau + 3c_{i,3}\tau^2 + \cdots + 7c_{i,7}\tau^6,$$

$$\ddot{p}_i(\tau) = 2c_{i,2} + 6c_{i,3}\tau + 12c_{i,4}\tau^2 + \cdots + 42c_{i,7}\tau^5.$$

In general, the derivative row used by the code is

$$\frac{d^r}{d\tau^r}\tau^k = k(k-1)\cdots(k-r+1)\tau^{k-r}.$$

Step 3: Minimum-Snap Optimization

The trajectory coefficients are selected by minimizing integrated squared snap:

$$\min_c \sum_i \int_0^{T_i} \left(\frac{d^4 p_i(\tau)}{d\tau^4} \right)^2 d\tau.$$

This becomes a quadratic program:

$$\min_c \frac{1}{2} c^\top Q c \quad \text{subject to} \quad Ac = b.$$

The code solves the equality-constrained KKT system:

$$\begin{bmatrix} Q & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} c \\ \lambda \end{bmatrix} = \begin{bmatrix} 0 \\ b \end{bmatrix}.$$

The constraints used in the current code are:

$$p_i(0) = w_i, \qquad p_i(T_i) = w_{i+1},$$

$$\dot{p}_0(0) = 0, \qquad \ddot{p}_0(0) = 0,$$

$$\dot{p}_{N-1}(T_{N-1}) = 0, \qquad \ddot{p}_{N-1}(T_{N-1}) = 0,$$

and at internal segment boundaries,

$$\dot{p}_i(T_i) = \dot{p}_{i+1}(0), \qquad \ddot{p}_i(T_i) = \ddot{p}_{i+1}(0).$$

So the path is smooth in velocity and acceleration at internal waypoints. The waypoints are not interpreted as stop points unless they are the first or final endpoint constraints.

Step 4: Compute x_d and u_d

At a sample time t, the code identifies the active segment and local time

$$\tau = t - \sum_{j<i} T_j.$$

Then it evaluates:

$$p_x(\tau), \quad p_y(\tau), \quad \dot{p}_x(\tau), \quad \dot{p}_y(\tau), \quad \ddot{p}_x(\tau), \quad \ddot{p}_y(\tau).$$

The desired state is

$$x_d(t) = \begin{bmatrix} p_x(t) \\ p_y(t) \\ \dot{p}_x(t) \\ \dot{p}_y(t) \end{bmatrix}.$$

The double-integrator model used by the LQR controller is

$$\dot{p} = v, \qquad \dot{v} = -dv + du, \qquad d = 1.1.$$

To realize a desired acceleration a_d, solve

$$a_d = -dv_d + du_d.$$

Therefore,

$$u_d = \frac{a_d + dv_d}{d} = v_d + \frac{a_d}{d}.$$

Per axis, the code computes

$$u_{x,d} = \frac{\ddot{p}_x + 1.1\dot{p}_x}{1.1}, \qquad u_{y,d} = \frac{\ddot{p}_y + 1.1\dot{p}_y}{1.1}.$$

Numerical Example: Calibration Sample 1858

This example uses row `sample_id=1858` from

`src/llm_vision_planner/fine_tuning/datasets/conformal_rrt_calibration_dataset_2001.csv`.

The raw LLM output for this row had two waypoints, but the refinement and verification pipeline produced three verified LLM waypoints. Minimum snap is run on the verified waypoints:

$$\begin{aligned} \text{RRT}: &\quad (1.51, 2.53),\ (0.83, 2.65),\ (0.47, 3.07), \\ \text{LLM}: &\quad (1.51, 2.53),\ (0.99, 2.63),\ (0.47, 3.07). \end{aligned}$$

The natural durations are:

$$\begin{aligned} T_{\text{RRT}} &= [1.3810,\ 1.1063], \\ T_{\text{LLM,separate}} &= [1.0591,\ 1.3624]. \end{aligned}$$

Under the shared-clock rule used in the calibration CSV, the LLM uses the RRT schedule:

$$T_{\text{LLM,shared}} = [1.3810,\ 1.1063].$$

Separate Clock

With separate clocks, RRT and LLM each receive durations from their own segment lengths. At the same wall-clock time $t = 0.6$ s:

$$\text{RRT first-segment progress} = \frac{0.6}{1.3810} = 43.4\%,$$

$$\text{LLM first-segment progress} = \frac{0.6}{1.0591} = 56.7\%.$$

So both are evaluated at the same time, but not at the same local phase of their first segment.

Trajectory	$x_d = [p_x, p_y, v_x, v_y]$ at $t = 0.6$	$u_d = [u_x, u_y]$ at $t = 0.6$
RRT	$[1.411,\ 2.519,\ -0.447,\ -0.026]$	$[-1.484,\ 0.082]$
LLM separate	$[1.366,\ 2.540,\ -0.619,\ 0.067]$	$[-1.820,\ 0.390]$

Using the same $dt = 0.1$ scoring step as the dataset generation script, the separate-clock comparison gives

$$s_u = 0.871456, \qquad s_x = 0.265008.$$

Shared Clock

With the shared-clock rule, RRT is generated normally, and the LLM min-snap trajectory is generated with the same segment durations as RRT. The LLM still uses minimum snap; only the time allocation changes.

At $t = 0.6$ s:

$$\text{RRT first-segment progress} = \text{LLM first-segment progress} = \frac{0.6}{1.3810} = 43.4\%.$$

Trajectory	$x_d = [p_x, p_y, v_x, v_y]$ at $t = 0.6$	$u_d = [u_x, u_y]$ at $t = 0.6$
RRT	$[1.411,\ 2.519,\ -0.447,\ -0.026]$	$[-1.484,\ 0.082]$
LLM shared	$[1.454,\ 2.514,\ -0.270,\ -0.048]$	$[-1.011,\ 0.023]$

For the calibration CSV row, the shared-clock score is

$$s_u = 0.836826, \qquad s_x = 0.268212.$$

Interpretation

Separate-clock scoring measures both path deviation and timing deviation:

$$\text{score} = \text{path mismatch} + \text{schedule mismatch} + \text{derivative amplification}.$$

Shared-clock scoring removes the schedule mismatch:

$$\text{score} = \text{path mismatch under a common execution schedule} + \text{derivative amplification}.$$

It does not remove derivative amplification. Since

$$u_d = v_d + \frac{a_d}{1.1},$$

small waypoint differences can still become larger control differences, especially for short durations. Shared-clock scheduling only prevents the LLM from being penalized because its first segment happened to receive a much shorter duration than the RRT segment.

The number of high-level waypoints does not increase. However, the number of sampled trajectory points changes with duration because the implementation uses

$$\text{samples per segment} = \left\lceil \frac{T_i}{dt} \right\rceil.$$

Plots

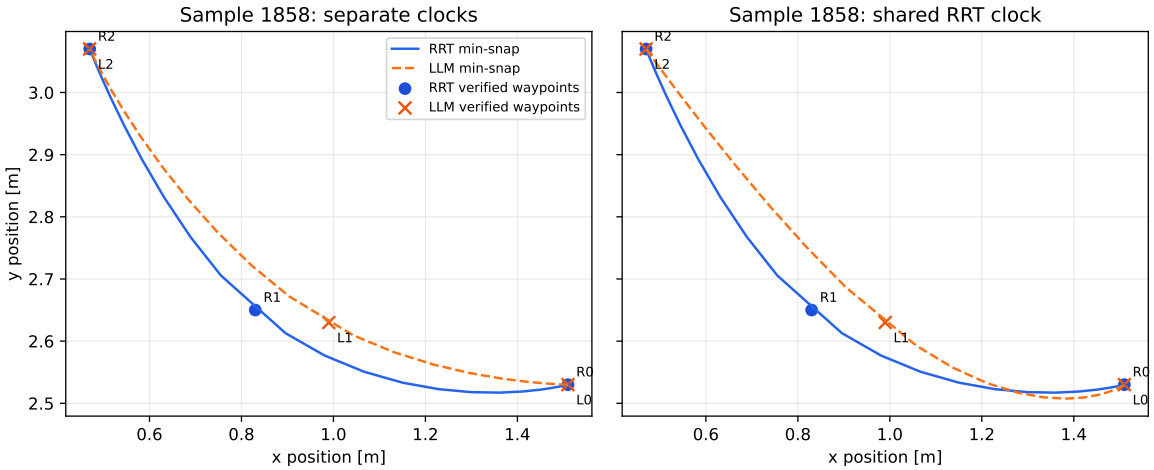

Figure 1: RRT and LLM waypoints with their generated minimum-snap paths under separate-clock and shared-clock scheduling.

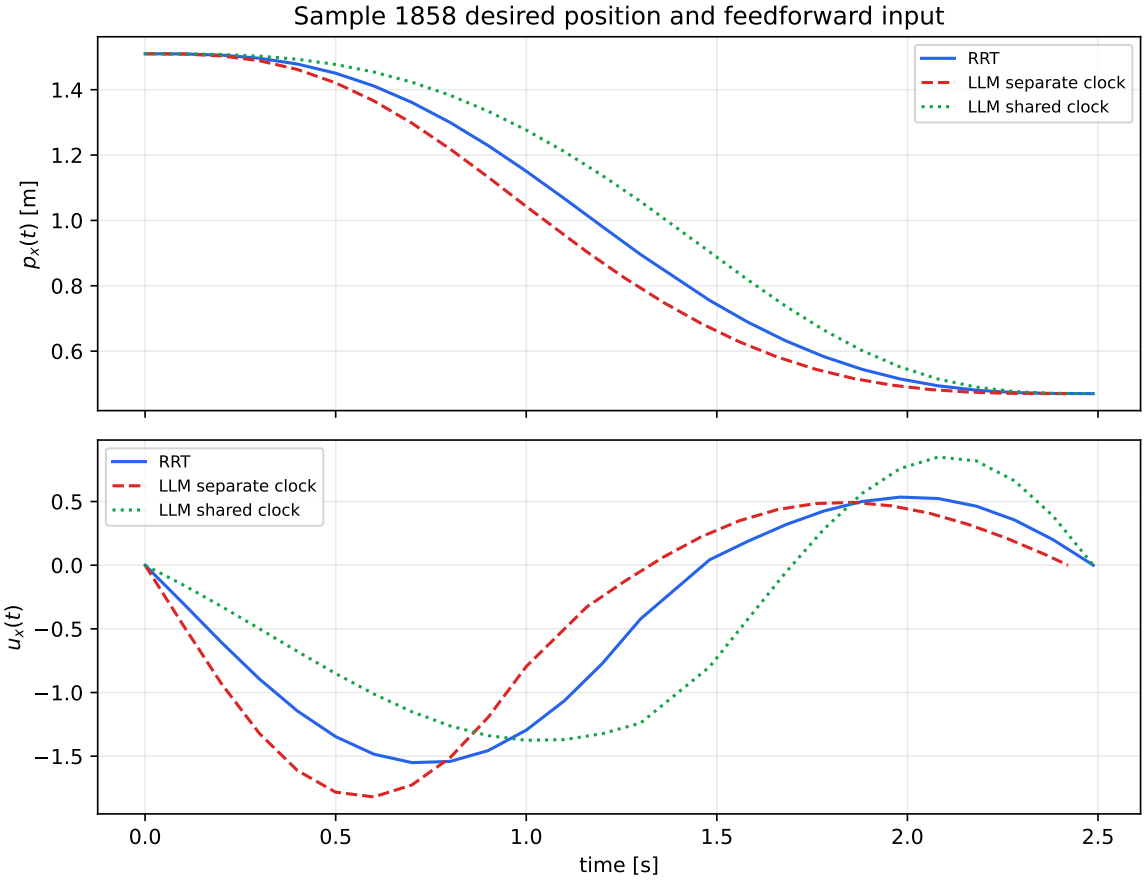

Figure 2: Desired x position and feedforward u_x over time. The separate LLM clock creates a larger early input spike because its first segment is much shorter.